

Homework 2: Reinforcement Learning Agent for CarRacing Environment

Zhuoyang Zou

This RL code, part of my master's course project and implemented by me and other team members, presents a reinforcement learning agent based on a Deep Q-Network (DQN) to play the 'CarRacing' game environment from OpenAI's Gymnasium library. (Here is our presentation link: <https://youtu.be/JKSpW0tA628>) The purpose of this project is to train an agent capable of autonomously driving a car in a 2D racing game by learning optimal actions through trial and error, aiming to maximize cumulative rewards.

The implementation leverages a convolutional neural network (CNN) as a Q-network to process the 2D game observations and predict action-value (Q) estimates, which guide the agent's decision-making. The Agent class is central to the reinforcement learning process, managing experience replay, epsilon-greedy action selection, and periodic updates to the target network, all of which are critical for stabilizing **DQN training**.

The overall approach involves initializing the environment and agent, gathering experiences through agent-environment interactions, and updating the Q-network via backpropagation on batches of past experiences. This training loop iteratively optimizes the agent's policy until it can reliably navigate the car to achieve high game scores.

1. QNetwork Class

This class defines the architecture of the neural network used to approximate the Q-value function. The Q-value function, $Q(s,a)$, estimates the expected reward of taking a particular action a in a given state s , and it is central to Q-learning.

- **Convolutional Layers:** The first few lines in this class define a series of convolutional layers. These layers extract spatial features from the game's images by applying convolution operations that detect edges, shapes, and patterns in the visual input.
- **Fully Connected Layers:** After flattening the output from the convolutional layers, the data passes through fully connected layers. These layers serve as decision-making layers, mapping the extracted features to Q-values for each possible action the agent can take.
- **Output Layer:** The final layer provides Q-values for each action, which are used by the agent to decide the best action based on the learned Q-values.

```

class QNetwork(nn.Module):
    def __init__(self):
        super(QNetwork, self).__init__()
        # Define convolutional layers for processing visual input
        self.conv1 = nn.Conv2d(3, 8, kernel_size=4, stride=2) # First conv layer
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, stride=2) # Second conv layer
        self.conv3 = nn.Conv2d(16, 32, kernel_size=3, stride=2) # Third conv layer
        self.conv4 = nn.Conv2d(32, 64, kernel_size=3, stride=2) # Fourth conv layer
        self.conv5 = nn.Conv2d(64, 128, kernel_size=3, stride=1) # Fifth conv layer
        self.conv6 = nn.Conv2d(128, 256, kernel_size=3, stride=1) # Sixth conv
layer

        # Fully connected layers to output Q-values for actions
        self.fc1 = nn.Linear(256 * 1 * 1, 100)
        self.fc2 = nn.Linear(100, 100)
        self.fc3 = nn.Linear(100, 5) # Output layer with 5 actions

    def forward(self, x):
        # Reorder input dimensions and apply relu activations through conv layers
        x = torch.permute(x, (0, 3, 1, 2))
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))
        x = F.relu(self.conv5(x))
        x = F.relu(self.conv6(x))
        # Flatten and pass through fully connected layers
        x = x.reshape(-1, 256 * 1 * 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x) # Q-values for each action

```

2. Agent Class

This class encapsulates the RL logic, defining how the agent interacts with the environment, stores experiences, and learns over time. Below are the main components and their significance:

- **Replay Memory:** The agent stores past experiences in a replay memory buffer. This buffer contains (state, action, reward, next_state, done) tuples, where:
 - **State:** The current observation of the environment.
 - **Action:** The action taken by the agent.
 - **Reward:** The immediate reward received after taking the action.
 - **Next State:** The new observation resulting from the action.
 - **Done:** A boolean flag indicating if the episode has ended.

Replay memory enables the agent to break the correlation between sequential experiences by sampling random batches of past experiences during training, leading to more stable and efficient learning.

- **Epsilon-Greedy Policy:** This policy balances exploration and exploitation, key concepts in RL:
 - **Exploration:** Trying new actions to discover their rewards.
 - **Exploitation:** Using known information to maximize rewards by selecting actions with the highest Q-values.
 - **Epsilon:** The exploration rate, gradually decaying over time, allows the agent to explore more at the beginning and exploit more as it learns.

With an epsilon-greedy policy, the agent occasionally selects a random action (exploration) but typically chooses the action with the highest Q-value (exploitation).

- **Target and Local Networks:** Two separate networks are used to improve the stability of Q-learning:
 - **Local Network:** Updated frequently to learn from the agent's interactions with the environment.
 - **Target Network:** Updated less frequently and helps the agent predict stable Q-values, reducing oscillations in training.
- **Discount Factor (γ gamma):** This parameter balances immediate and future rewards, impacting the agent's focus on short-term versus long-term gains.

3. Key Methods in Agent Class

- **act():** This function determines the agent's actions. It uses the epsilon-greedy policy to either:
 - Select a random action for exploration.
 - Use the local network to predict Q-values and select the action with the highest Q-value for exploitation.
- **step():** This function manages experience storage and learning:
 - **Storing Experiences:** Each experience is stored in replay memory.

- **Periodic Training:** Every few steps, the agent samples a batch of experiences from memory and calls `learn()` to update the Q-network using these experiences.
- **learn():** This function implements the core Q-learning update step:
 - **Predict Q-values:** It calculates predicted Q-values for current states and actions from the local network.
 - **Target Q-values:** It computes target Q-values using rewards and the maximum Q-value of the next state (from the target network).
 - **Loss Calculation:** The loss is the difference between predicted and target Q-values, computed using mean squared error (MSE).
 - **Backpropagation:** The optimizer adjusts the weights in the local network to minimize this loss, thus improving Q-value predictions.

```
class Agent():
    def __init__(self):
        # Initialize two networks for stable Q-learning
        self.qnetwork_local = QNetwork().to(device)
        self.qnetwork_target = QNetwork().to(device)

        # Replay memory to store past experiences
        self.memory = deque(maxlen=10000)

        # RL hyperparameters
        self.gamma = 0.97          # Discount factor
        self.epsilon = 1.0         # Initial exploration rate
        self.epsilon_min = 0.01    # Minimum exploration rate
        self.epsilon_decay = 0.9999 # Decay rate for epsilon
        self.batch_size = 512
        self.train_start = 3000    # Start training after this many experiences

        # Optimizer
        self.optimizer = optim.Adam(self.qnetwork_local.parameters(), lr=0.001)

    def remember(self, state, action, reward, next_state, done):
        # Store the experience in memory
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        # Choose action based on epsilon-greedy policy
```

```

state = torch.from_numpy(state).float().unsqueeze(0).to(device)
self.qnetwork_local.eval()
with torch.no_grad():
    action_values = self.qnetwork_local(state)
self.qnetwork_local.train()

# If random sample exceeds epsilon, choose best action; otherwise, random
if random.random() > self.epsilon:
    return np.argmax(action_values.cpu().data.numpy())
else:
    return random.choice(np.arange(5))

def step(self, state, action, reward, next_state, done):
    # Store experience and perform periodic training updates
    self.remember(state, action, reward, next_state, done)

    # Update target network periodically
    if len(self.memory) >= self.train_start and random.random() < 0.25:
        self.learn(random.sample(self.memory, self.batch_size))

def learn(self, batch):
    # Learn from a batch of experiences
    criterion = torch.nn.MSELoss()

    # Preprocess batch
    states, actions, rewards, next_states, dones = map(np.array, zip(*batch))
    states = torch.tensor(states, dtype=torch.float32, device=device)
    actions = torch.tensor(actions, dtype=torch.int64, device=device)
    rewards = torch.tensor(rewards, dtype=torch.float32, device=device)
    next_states = torch.tensor(next_states, dtype=torch.float32, device=device)
    dones = torch.tensor(dones, dtype=torch.float32, device=device)

    # Get Q predictions and targets
    predictions = self.qnetwork_local(states).gather(1, actions.unsqueeze(-1)).squeeze(-1)
    with torch.no_grad():
        next_q_values = self.qnetwork_target(next_states).max(1)[0]
        targets = rewards + (self.gamma * next_q_values * (1 - dones))

    # Compute loss and update local network

```

```
loss = criterion(predictions, targets)
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
```

Here is the video demo:

